

פרויקט תכנותי במבני נתונים

מטרות הפרויקט:

1. העמקת הבנת נושאי הקורס התאורטיים שנלמדו. בפרט, בחינת תוצאות תאורטיות שנלמדו בכיתה באופן אמפירי וביקורתי כדי להעשיר את תפיסת הסטודנטים לגביהן.
2. שיפור מיומנויות הקשורות למשימות תכנות בעידן ה LLMs: פחות כתיבת קוד מאפס, יותר קריאה, עריכה, ניתוח, טסטינג, השוואה, הערכה וכו' של קוד נתון (למשל כזה שהתקבל ממודלי שפה).
3. שיפור מיומנות לימוד עצמי – הן של חומר תאורטי והן של שימוש בספריות קוד לא מוכרות.

הערה: הגם שזו אינה מטרה מוצהרת של הפרויקט, הוא מהווה הזדמנות לבצע פרויקט מעשי משמעותי ורלוונטי בשלב מוקדם בתואר. סטודנטים משנים מתקדמות מדווחים שפרויקטים כאלו מועילים למשל בראיונות עבודה, בשלב בו לסטודנטים אין עדיין "תיק עבודות" משמעותי להציג. נקודת מבט זו נלקחה בחשבון בעת תכנון פרויקט זה ע"י צוות הקורס בסמסטר ב' תשפ"ו.

הוראות הגשה

הגשת התרגיל תתבצע באופן אלקטרוני באתר הקורס במודל. המלצתנו היא להגיש בזוגות אך ניתן גם לעבוד לבד. נפתח פורום ייעודי למציאת בני זוג במודל. כל זוג יבחר נציג/ה ויעלה רק תחת שם המשתמש של הנציג/ה את קבצי התרגיל (תחת קובץ zip) למודל.

על ההגשה לכלול שלושה קבצים:

1. קובץ קוד (הרחבה של קובץ השלד שניתן) תחת השם **AVLTree.py** עם המימוש של חלק א' בפרוייקט.
2. מסמך תשובות בפורמט pdf, המכיל תשובות לחלק ב' בפרוייקט.
3. קובץ טקסט **info.txt** המכיל את פרטי הזוג: שמות מלאים בעברית, שמות מלאים באנגלית, מספרי ת"ז ואימיילים אוניברסיטאיים של המגישים.

שמות קובץ התשובות וקובץ הק zip צריכים לכלול את שמות המשתמש האוניברסיטאיים של הזוג המגיש לפי הפורמט **AVLTree_username1_username2.pdf/zip**, בתוכן הקבצים יש לציין את שמות המשתמש, תעודות הזהות ושמות המגישים (בכותרת המסמך ובשורת הערה בקובץ המקור).

הגשת שיעורי הבית באיחור - באישור מראש בלבד. הגשה באיחור ללא אישור תגרור הורדת נקודות מהציון.

בהצלחה!

חלק א – מימוש בקוד

בחלק זה תממשו עץ AVL. מותר, אך לא חובה, להיעזר במודלי שפה (LLM, למשל Claude, ChatGPT ואחרים) בכל דרך שתירצו. בסוף מסמך זה ריכזנו עבורכם כמה הכוונות ועצות מועילות לשימוש במודלי שפה במשימות תכנות (שבתקווה יועילו לכם גם בהמשך לימודיכם ובעתיד).

דרישות

בתרגיל זה יש לממש עץ חיפוש בינארי (BST) ועץ AVL לפי ההגדרות שניתנו בכיתה. המימוש יהיה משותף והקביעה האם העץ יהיה עץ חיפוש בינארי "רגיל" או עץ AVL תהיה תלויה בפרמטר בוליאני שיסופק בבנאי. לכל איבר בעץ יש ערך (value) שהוא מחרוזת, ומפתח (key) שהוא מספר שלם. כל המפתחות שונים זה מזה, והסדר על צמתי העץ מתייחס כרגיל אך ורק למפתחות. המימוש יהיה בשפת python 3.14.5 וצריך להיות מבוסס על קובץ השלד המופיע באתר הקורס.

להלן הפעולות שיש לממש. שימו לב: על כל הפעולות לתמוך בזמני ריצה כפי שניתחנו בכיתה, הן במקרה הגרוע והן לשיעורין (בנוגע לעלות התיקונים אחרי הכנסה), ואם הפעולה לא הופיעה בכיתה אז בסיבוכיות הזמן הטובה ביותר האפשרית:

פעולה	תיאור
init(is_avl)	הבנאי של המחלקה מקבל פרמטר בוליאני is_avl שקובע את התנהגות המבנה. אם is_avl=True אז מדובר בעץ AVL, אחרת (is_avl=False) מדובר בעץ חיפוש בינארי (BST), כלומר לא יופעלו בו מנגנוני האיזון (רוטציות).
search(k)	הפונקציה מחפשת איבר בעל מפתח k ומחזירה את הזוג (x, search_time) כאשר: x הוא מצביע לצומת המתאים - אם קיים, אחרת None. פירוט לגבי החישוב של search_time יופיע בהמשך.
insert(k, s)	הכנסת איבר בעל מפתח k וערך s לעץ. ניתן להניח שהמפתח k לא קיים בעץ בעת הקריאה לפעולה. הפונקציה מחזירה את הרביעייה (x, search_time, rotations, height_changes), כאשר x זה מצביע לצומת שהוכנס. על שלושת הערכים המוחזרים הנוספים נסביר בהמשך.
delete(x)	מחיקת הצומת x מהעץ בהינתן מצביע.
avl_to_list()	הפונקציה מחזירה רשימה ממוינת (ע"פ המפתחות) של האיברים במילון כאשר כל איבר מיוצג ע"י זוג סדור של (key, value).
size()	הפונקציה מחזירה את מספר האיברים בעץ.
get_root()	הפונקציה מחזירה מצביע לשורש העץ (אם העץ ריק אז מחזירה None).
get_height()	הפונקציה מחזירה את גובה העץ (אם העץ ריק אז מחזירה -1). בעץ AVL זמן הריצה של הפעולה יהיה O(1). בעץ חיפוש בינארי זמן הריצה יהיה O(n).

לצורך מימוש פעולות אלו, ניעזר במחלקה **AVLNode** המופיעה בקובץ. מטעמי נוחות, **נדרש** שלכל עלה יהיו 2 בנים "וירטואליים", כלומר צמתים ללא מפתח. באופן זה, נוח יותר לממש גלגולים מכיוון שמבחינה מבנית לכל צומת יהיו 2 בנים. מטעמי חיסכון במקום יהיה צומת וירטואלי **יחיד** שמשותף לכל העץ.

המחלקה AVLNode מכילה את השדות הבאים:

- key – המפתח של הצומת.
 - value – הערך של הצומת.
 - left – הבן השמאלי של הצומת.
 - right – הבן הימני של הצומת.
 - parent – ההורה של הצומת.
 - height – גובה הצומת (יש לתחזק את השדה הזה רק עבור עץ AVL).
- בנוסף, המחלקה תומכת בפעולה הבאה:
- is_real_node – מחזירה True אם ורק אם הצומת מייצג צומת אמיתי בעץ (קרי: צומת שאינו וירטואלי).

הסבר לגבי החישוב של זמני חיפוש

בפעולות insert וsearch אנו מבצעים חיפוש בעץ. בפעולה insert, מטרת החיפוש היא למצוא את מקום ההכנסה ומכיוון שמניחים שהמפתח שרוצים להכניס לא נמצא בעץ טרם הקריאה לפעולה - החיפוש הזה יהיה בהכרח חיפוש כושל.

נגדיר את הביטוי search_time שיש להחזיר בפעולות insert וsearch:

- עבור עץ ריק נגדיר $search_time=1$.
- עבור חיפוש **מוצלח** בעץ לא ריק $search_time$ שווה לאורכו של המסלול הפשוט⁽¹⁾ בעץ בין הצומת ההתחלתי לצומת שחיפשנו ומצאנו בהצלחה+1.
- שימו לב שהביטוי הנ"ל שווה למספר הצמתים במסלול מהשורש לצומת.
- עבור חיפוש **כושל** בעץ לא ריק $search_time$ שווה לאורכו של המסלול הפשוט בעץ בין הצומת ההתחלתי לצומת האמיתי האחרון שהגענו אליו בחיפוש בעץ+2.
- שימו לב שהביטוי הזה שווה ל- $search_time$ שהיה מתקבל אילו המפתח שחיפשנו היה עלה בעץ.

דוגמאות לערכי search_time שיש להפיק מופיעות בטסטר שסיפקנו לכם עם התרגיל.

הסבר לגבי החישוב של זמני האיזון של העץ

בפעולות insert בעץ AVL, לאחר ההכנסה של עץ חיפוש בינארי, יש לאזן את העץ כפי שלמדנו בכיתה (גם בפעולות delete יש לאזן את העץ אבל במטלה הזו אנו מודדים את זמני האיזון של העץ רק עבור הכנסות). הערכים rotations, height_changes שאותם יש להחזיר מהווים מדד לזמן הריצה של פעולות האיזון והם יוגדרו כדלקמן:

rotations יוגדר להיות מספר הגלגולים שבוצעו (גלגול כפול נספר כ-2 גלגולים).

height_changes סופר את מספר "פעולות שינוי הגובה" – מונה של מקרה 3.3 באלגוריתם ההכנסה לעץ AVL, כפי שמופיע בשקף שלהלן. אנו לא סופרים שינויי גובה של צומת חדש שהוכנס לעץ ולא סופרים שינויי גובה בתוך גלגולים.

⁽¹⁾ מסלול פשוט הוא מסלול שלא עובר בצומת יותר מפעם אחת

The Insertion Algorithm

The Algorithm

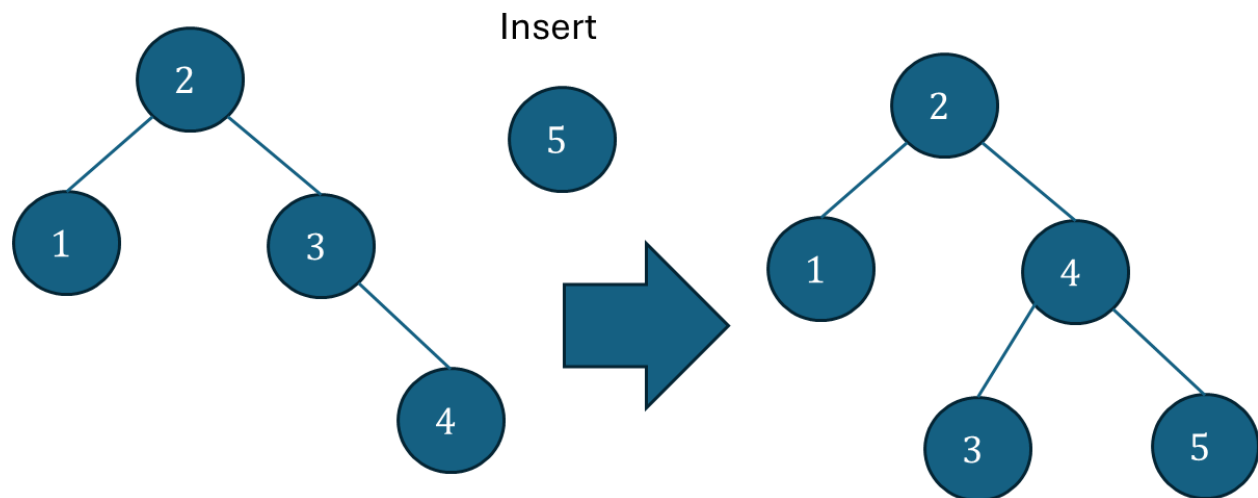
AVL-Insert(T, z)

1. insert z as usual (as in a BST)
2. let y be the parent of the inserted node.
3. while $y \neq \text{Null}$ do:
 - 3.1. compute $BF(y)^*$
 - 3.2. if $|BF(y)| < 2$ and y 's height hasn't changed: terminate Obs. 3
 - 3.3. else if $|BF(y)| < 2$ and y 's height changed: go back to stage 3 with y 's parent Obs. 2
 - 3.4. else ($|BF(y)| = 2$): perform a rotation and go back to stage 3 with y 's parent Obs. 1

*Requires maintaining additional information at each node. We will refer to this topic later.

דוגמה קטנה: סדרת הכנסה של 3, 1, ואז 2. בהכנסה הראשונה אין שינוי גובה. בהכנסה של 1 מעדכנים את הגובה של 3 (מקרה 3.3): פעולת שינוי גובה אחת. בהכנסה של 2 מעדכנים את הגובה של 1 (מקרה 3.3), כלומר פעולת שינוי גובה אחת, ולאחר מכן בשורש 3 אנחנו נמצאים במקרה 3.4 של גלגול כפול, מה שלא נחשב כפעולת שינוי גובה אבל כן נחשב כ-2 גלגולים בספירה של rotations.

דוגמה נוספת: נניח שאנחנו מכניסים את 5 לעץ שמשמאל. הגובה של 4 מעודכן (מקרה 3.3): פעולת שינוי גובה אחת. לאחר מכן מבצעים גלגול על הקשת שמחברת את 3 ו-4, מה שלא מהווה פעולת שינוי גובה אבל כן מהווה גלגול אחד בספירה של rotations.



הערות חשובות:

1. המימוש יבוצע על ידי מילוי קובץ ה"שלד" שפורסם עם התרגיל.
2. אסור לשנות את הגדרות הפונקציות הקיימות בשלד ואסור למחוק שדות קיימים כי זה יפגע בטסטר. על כל הפונקציות/מחלקות להופיע בקובץ יחיד.
3. במידת הצורך, ניתן להרחיב את המימוש: ניתן להוסיף פונקציות עזר חדשות שאינן מופיעות בשלד, להוסיף שדות חדשים או לשנות את האיתחול של שדות קיימים. מותר גם להוסיף לפונקציות קיימות פרמטרים נוספים עם ערכי ברירת מחדל, בצורה זו הטסטר לא נפגע.
4. אין להשתמש באף מימוש ספרייה של מבנה נתונים (כמו עץ חיפוש).
5. עליכם לממש את כל הפעולות בסיבוכיות המיטבית.
6. אין צורך בפלט למשתמש. בפרט, בקובץ שתגישו לא תהיה פונקציית **main** ולא יהיו הרצות קוד/הדפסות, דבר זה יפגע בטסטר שיבדוק לכם את התרגילים.

בדיקות

התרגילים ייבדקו באמצעות תוכנת טסטר שקוראת לפונקציות המפורטות מעלה בתרחישים שונים, ומוודאת את נכונות התוצאות. קובץ הטסטר שלנו לא יפורסם, אך תפורסם גרסה מצומצמת שלו על מנת לאפשר לכם לבצע בדיוק בסיסיות. מומלץ מאוד לממש אוסף בדיקות מקיף יותר (אך זיכרו להסיר אותן לפני ההגשה) כדי לבדוק שהקוד לא רק רץ, אלא גם נכון!

חלק ב – ניסויים וחלק תיאורטי

הערות:

- אין לכלול את הקוד שהרצתם עבור הניסויים בקובץ py שאתם מגישים (הוא יכול להופיע שם אבל מוסתר בהערה).
- לצורך הצגת תוצאות באיורים (גרפים) ניתן להשתמש בכל כלי שתמצאו, למשל Excel. יחד עם זאת ההמלצה שלנו היא להשתמש בספריה כמו matplotlib של פייתון שמאפשרת ויזואליזציה של נתונים בקלות. **איננו מצפים שתלמדו להפעיל את הספריה בעצמכם - אפשר ואף מומלץ להיעזר במודלי שפה**, כי הקוד הדרוש לצורך כך הוא בעיקר טכני. תפעול קוד מספריות שלא מוכרות לכם מתאפשר כיום בקלות יחסית הודות לשימוש במודלי שפה. כמובן, באחריותכם לוודא שהאיורים תקינים, נכונים וברורים.

בהינתן n טבעי, נגדיר ארבעה ניסויים:

ניסוי 1: נאתחל עץ חיפוש בינארי ריק ואז נכניס אליו את המפתחות $1, 2, \dots, n$ בסדר ממויין.

ניסוי 2: כמו ניסוי 1 אבל עם עץ AVL במקום עץ חיפוש בינארי.

ניסוי 3: נאתחל עץ חיפוש בינארי ריק ואז נכניס אליו את האיברים $1, 2, \dots, n$ בסדר אקראי (כל פרמוטציה של $1, 2, \dots, n$ שקובעת את סדר ההכנסה מוגרלת בהסתברות שווה).

ניסוי 4: כמו ניסוי 3 אבל עם עץ AVL במקום עץ חיפוש בינארי.

שאלה 1:

נתחו אסימפטוטית, כתלות ב n , את זמן הריצה של ביצוע ניסוי 1 וגם את עומק העץ שמתקבל בסוף ניסוי זה.

שאלה 2:

חזרו על שאלה 1 עבור ניסוי 2.

שאלה 3:

יש לבצע את כל אחד מארבעת הניסויים עבור כל אחד מערכי ה- $n = 300 \cdot 2^i$ כאשר $i = 1, 2, \dots, 10$. עבור כל ניסוי מלאו את הטבלה הבאה וצרו גרף של זמן הריצה (במילישניות) כתלות ב- n .

הערה: בניסויים האקראיים (ניסויים 3 ו-4), עבור כל n , יש לבצע 20 ניסויים (כלומר יש להגריל 20 פרמוטציות של $1, 2, \dots, n$, ועבור כל אחת מהן לבצע את הניסוי). יש להתחשב בממוצעים של הניסויים האלו בטבלאות שתמלאו ובגרפים שתיצרו.

גודל העץ n	גובה העץ	מספר הגלגולים	סה"כ שינויי הגובה	זמן ריצה (בפעולות)	זמן ריצה (במילישניות)
...	...			...	

זמן הריצה (בפעולות) של ניסוי שווה לסכום של שלושת הגורמים הבאים:

- סה"כ זמני הריצה של החיפוש שבוצעו בניסוי (search_time שמוחזר ב- insert או ב- search).
 - סה"כ מספר הגלגולים.
 - סה"כ מספר שינויי הגובה.
- נזכיר שבעץ חיפוש בינארי "רגיל" גורמים 2 ו-3 שווים ל-0 (כי לא מבצעים פעולות איזון). בנוסף, בעץ כנ"ל זמן הריצה של חישוב גובה העץ שמתקבל (בסוף הניסוי) מתבצע בזמן $O(n)$. אנו מתעלמים מהעלות הזו כשאנו דנים בזמן הריצה (בפעולות) של ניסוי. שימו לב שאסימפטוטית זה לא משנה כי כל ניסוי ממילא רץ בזמן $\Omega(n)$.

שאלה 4:

א. חוו דעתכם על התוצאות שקיבלתם בשאלה 3 עבור ארבעת הניסויים. בפרט, התייחסו בנפרד לגובה העץ שמתקבל, למספר הגלגולים ולסה"כ שינויי הגובה.

ב. עבור ניסויים 1 ו-2 – השוו זאת לתשובות שעניתם בשאלות 1 ו-2.

ג. בכיתה ראינו שתי טענות:

- בהכנסה לעץ AVL מתבצע לכל היותר גלגול אחד (רגיל או כפול).
 - בעץ AVL בסדרה של הכנסות בלבד העלות לשיעורין של האיזון בפעולת ההכנסה היא קבועה.
- האם הטענות האלו באות לידי ביטוי בתוצאות הניסויים? הסבירו.

שאלה 5:

אם סדרת הפעולות יכולה להכיל פעולות מחיקה אז המסקנה של טענה (II) לא תקפה: לא בהכרח מתקיים שהעלות לשיעורין של האיזון בפעולת ההכנסה היא קבועה.

א. הוכיחו זאת.

ב. גבו זאת בניסוי מתאים.

בהצלחה!

שימוש במודלי שפה

מטרת חלק זה במסמך היא לתת **עצות כלליות והכוונה** לשימוש נכון ויעיל במודלי שפה (LLMs – Large Language Models) כדוגמת ChatGPT או Claude. אופן הפעולה של כלים אלו אינו כלול בחומר הקורס והוא נושא שנלמד בקורסי בחירה מתקדמים כמו עיבוד שפה טבעית. יחד עם זאת השימוש בהם פשוט ונגיש לכל, וחשוב להבין את היכולות ובמיוחד את המגבלות שלהם (נכון להיום).

בראש ובראשונה, זיכרו שמודלי שפה, גם המתקדמים ביותר, **עלולים לטעות ולהטעות**. בפרט, הם עלולים לכתוב קוד שגוי. למשל קוד שלא עובד בכלל, קוד שנופל בזמן ריצה, קוד שלא עומד בדרישות, קוד לא יעיל וכו'. תופעה ידועה בהקשר זה היא **"הזיות"** (hallucinations) - תשובות שנשמעות משכנעות אך מכילות מידע שגוי לגמרי (בהקשר התכנותי, זה יכול להיות למשל שימוש בספריות או פונקציות שלא קיימות). כמו כן, מודלי שפה טועים לעיתים קרובות בניתוח סיבוכיות זמן וזיכרון, או מציעים פתרון שנראה נכון אך אינו עומד בדרישות הסיבוכיות של התרגיל. תמיד נתחו בעצמכם את הסיבוכיות של כל פעולה והשוו אותה לדרישות.

מעבר לכך, כדי שתהליך הלמידה שלכם יהיה יעיל ומועיל, הקפידו תמיד להבין ולא סתם להשתמש בתוצרים של מודלי שפה. **בסופו של דבר, מה שאתם מגישים הוא באחריות שלכם, גם אם נעזרתם במודלי שפה**. זכרו: מודלי שפה הם כלי עזר ללמידה ופיתוח, לא תחליף להבנה מעמיקה של החומר.

הנה עוד כמה עצות:

- כאמור, הבדיקה של התוצרים היא קריטית - תמיד יש לבדוק את הקוד שנוצר, טיפול במקרי קצה, עמידה בדרישות, וכו'.
- כדאי להגדיר במדויק את הפורמט שאתם מצפים לקבל כתשובה. למשל "תן לי את התשובה בפורמט של פסאודו-קוד ואז מימוש בפייטון עם הערות", או "ממש את הפונקציה ואז הסבר את אופן פעולתה בכמה משפטים".
- פירוק בעיות – מודלי שפה נוטים להיות כיום טובים יותר במשימות קטנות. כדאי לשקול לחלק את הבעיה לתת בעיות, במקום לבקש פתרון שלם לבעיה כולה.
- אפשר לכתוב טסטים לבדיקת נכונות הקוד בעצמכם, ואפשר גם לבקש ממודלי שפה לספק טסטים, או להציע מקרים שאליהם כדאי להתייחס בטסטים וכו'. יחד עם זאת תמיד יש להריץ בעצמכם את הבדיקות ולחשוב על מקרים נוספים שכדאי לבדוק.
- אפשר לבקש ממודלי שפה לא רק לכתוב קוד, אלא גם להסביר אותו, לפרט את השלבים, לכתוב אותו בפסאודו קוד. אפשר לתת למודלי שפה קוד קיים ולבקש הסבר, שיפור, השוואה לקוד אחר או בין גישות שונות וכו'.
- אפשר לאתגר מודלי שפה בשאלות כמו "מה החיסרון בפתרון זה?" או "האם שתי השורות הראשונות בפונקציה מטפלות בכל מקרי הקצה?"
- זיכרו שמודלי שפה נוטים "לרצות" (to please), וזאת בשל האופן בו אומנו. זה אומר שלפעמים אופן ניסוח השאלה יכול להטות את התשובה. למשל אם תשאלו "מה הבאג בקוד?" ייתכן שהמודל ימצא באג גם אם אין כזה.